Set - 01

Q1. Write a C program to find the total and average of n students using structures.

```
#include <stdio.h>

struct Student {
   char name[50];
   float marks;
};

int main() {
   int n, i;
   float total = 0, average;

   printf("Enter number of students: ");
   scanf("%d", &n);

   struct Student s[n];

   for(i = 0; i < n; i++) {
      printf("\nEnter name of student %d: ", i + 1);
      scanf("%s", s[i].name);

      printf("Enter marks: ");
      scanf("%f", &s[i].marks);

      total += s[i].marks;
   }
```

```
    average = total / n;

    printf("\nTotal Marks = %.2f", total);
    printf("\nAverage Marks = %.2f", average);

    return 0;
}
```

Q2. Implement Merge Sort using arrays.

```
#include <stdio.h>

void merge(int arr[], int left, int mid, int right) {
    int i = left, j = mid + 1, k = 0;
    int temp[100];

    while (i <= mid && j <= right) {
        if (arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= right)
        temp[k++] = arr[j++];
```

```
    for (i = left, k = 0; i <= right; i++, k++)
        arr[i] = temp[k];
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);
```

```
    mergeSort(arr, 0, n - 1);

    printf("Sorted array:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Set - 02

Q1. Write a C program to copy one structure variable to another structure of the same type.

```
#include <stdio.h>

struct Student {
    int id;
    char name[50];
    float marks;
};

int main() {
    struct Student s1 = {101, "Ravi", 85.5};
    struct Student s2;

    // Copy structure
    s2 = s1;

    printf("Details of Student 1:\n");
```

```
    printf("ID: %d\nName: %s\nMarks: %.2f\n", s1.id, s1.name, s1.marks);

    printf("\nDetails of Student 2 (Copied):\n");
    printf("ID: %d\nName: %s\nMarks: %.2f\n", s2.id, s2.name, s2.marks);

    return 0;
}
```

Q2. Implement Quick Sort using arrays.

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];  // choose last element as pivot
    int i = low - 1;
    int j;

    for (j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
```

```
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    quickSort(arr, 0, n - 1);

    printf("Sorted array:\n");
```

```
  for (i = 0; i < n; i++)
    printf("%d ", arr[i]);

  return 0;
}
```

Set - 03

Q1. Write a C program to read student name and marks from the command line and display the student details along with the total.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
  int i;
  float marks, total = 0;

  if (argc < 3) {
    printf("Usage: %s <Name> <Marks1> <Marks2> ...\n", argv[0]);
    return 1;
  }

  printf("Student Name: %s\n", argv[1]);

  printf("Marks:\n");
  for (i = 2; i < argc; i++) {
    marks = atof(argv[i]);  // convert string to float
    printf("Subject %d: %.2f\n", i - 1, marks);
    total += marks;
  }
```

```
    printf("Total Marks = %.2f\n", total);

    return 0;
}
```

Q2. Implement Radix Sort using arrays.

```
#include <stdio.h>

int getMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > max)
            max = arr[i];
    return max;
}

void countSort(int arr[], int n, int exp) {
    int output[n];
    int count[10] = {0};

    // Count occurrences
    for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    // Cumulative count
    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];
```

```
    // Build output array
    for (int i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    // Copy back
    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}

void radixSort(int arr[], int n) {
    int max = getMax(arr, n);

    for (int exp = 1; max / exp > 0; exp *= 10)
        countSort(arr, n, exp);
}

int main() {
    int n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
```

```
    scanf("%d", &arr[i]);

  radixSort(arr, n);

  printf("Sorted array:\n");
  for (i = 0; i < n; i++)
    printf("%d ", arr[i]);

  return 0;
}
```

Set - 04

Q1. Implement Stack and its operations (Push, Pop, Peek, Display) using arrays.

```
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

// Push operation
void push(int value) {
  if (top == MAX - 1) {
    printf("Stack Overflow\n");
  } else {
    stack[++top] = value;
    printf("%d pushed into stack\n", value);
  }
}
```

```
// Pop operation
void pop() {
   if (top == -1) {
      printf("Stack Underflow\n");
   } else {
      printf("%d popped from stack\n", stack[top--]);
   }
}

// Peek operation
void peek() {
   if (top == -1) {
      printf("Stack is empty\n");
   } else {
      printf("Top element is %d\n", stack[top]);
   }
}

// Display operation
void display() {
   if (top == -1) {
      printf("Stack is empty\n");
   } else {
      printf("Stack elements:\n");
      for (int i = top; i >= 0; i--) {
         printf("%d\n", stack[i]);
      }
   }
```

```
}

int main() {
   int choice, value;

   while (1) {
      printf("\n1.Push 2.Pop 3.Peek 4.Display 5.Exit\n");
      printf("Enter choice: ");
      scanf("%d", &choice);

      switch (choice) {
         case 1:
            printf("Enter value: ");
            scanf("%d", &value);
            push(value);
            break;

         case 2:
            pop();
            break;

         case 3:
            peek();
            break;

         case 4:
            display();
            break;
```

```
        case 5:
            return 0;

        default:
            printf("Invalid choice\n");
      }
   }

   return 0;
}
```

Q2. Perform different operations (Insert, Delete, Display) in a Double Linked List.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *prev, *next;
};

struct Node *head = NULL;

// Insert at beginning
void insert() {
   int value;
   printf("Enter value: ");
   scanf("%d", &value);
```

```
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
        head->prev = newNode;

    head = newNode;
}

// Delete from beginning
void delete() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;
    printf("%d deleted\n", temp->data);

    head = head->next;

    if (head != NULL)
        head->prev = NULL;

    free(temp);
}
```

```
// Display list
void display() {
   struct Node *temp = head;

   if (temp == NULL) {
      printf("List is empty\n");
      return;
   }

   printf("Elements: ");
   while (temp != NULL) {
      printf("%d ", temp->data);
      temp = temp->next;
   }
   printf("\n");
}

int main() {
   int choice;

   while (1) {
      printf("\n1.Insert  2.Delete  3.Display  4.Exit\n");
      printf("Enter choice: ");
      scanf("%d", &choice);

      switch (choice) {
         case 1:
```

```
            insert();
            break;

        case 2:
            delete();
            break;

        case 3:
            display();
            break;

        case 4:
            return 0;

        default:
            printf("Invalid choice\n");
      }
   }

   return 0;
}
```

Set - 05

Q1. Implement Queue and its operations (Enqueue, Dequeue, Display) using arrays.

```
#include <stdio.h>
#define MAX 100

int queue[MAX];
```

```
int front = -1, rear = -1;

// Enqueue operation
void enqueue(int value) {
   if (rear == MAX - 1) {
      printf("Queue Overflow\n");
   } else {
      if (front == -1)
         front = 0;
      queue[++rear] = value;
      printf("%d inserted into queue\n", value);
   }
}

// Dequeue operation
void dequeue() {
   if (front == -1 || front > rear) {
      printf("Queue Underflow\n");
   } else {
      printf("%d deleted from queue\n", queue[front++]);
   }
}

// Display operation
void display() {
   if (front == -1 || front > rear) {
      printf("Queue is empty\n");
   } else {
```

```
        printf("Queue elements:\n");
        for (int i = front; i <= rear; i++) {
            printf("%d ", queue[i]);
        }
        printf("\n");
    }
}

int main() {
    int choice, value;

    while (1) {
        printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
```

```
            display();
            break;

        case 4:
            return 0;

        default:
            printf("Invalid choice\n");
      }
   }

   return 0;
}
```

Q2. Perform different operations (Insert, Delete, Display) in a Circular Linked List.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

struct Node *head = NULL;

// Insert at beginning
void insert() {
   int value;
   printf("Enter value: ");
```

```
    scanf("%d", &value);

    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;

    if (head == NULL) {
        newNode->next = newNode;
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != head)
            temp = temp->next;

        newNode->next = head;
        temp->next = newNode;
        head = newNode;
    }
}

// Delete from beginning
void delete() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;
```

```
    if (head->next == head) {
        printf("%d deleted\n", temp->data);
        free(temp);
        head = NULL;
    } else {
        struct Node *last = head;
        while (last->next != head)
            last = last->next;

        printf("%d deleted\n", temp->data);
        head = head->next;
        last->next = head;
        free(temp);
    }
}

// Display list
void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;
    printf("Elements: ");
    do {
        printf("%d ", temp->data);
        temp = temp->next;
```

```
    } while (temp != head);
    printf("\n");
}

int main() {
    int choice;

    while (1) {
        printf("\n1.Insert  2.Delete  3.Display  4.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                insert();
                break;

            case 2:
                delete();
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;
```

```
            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```